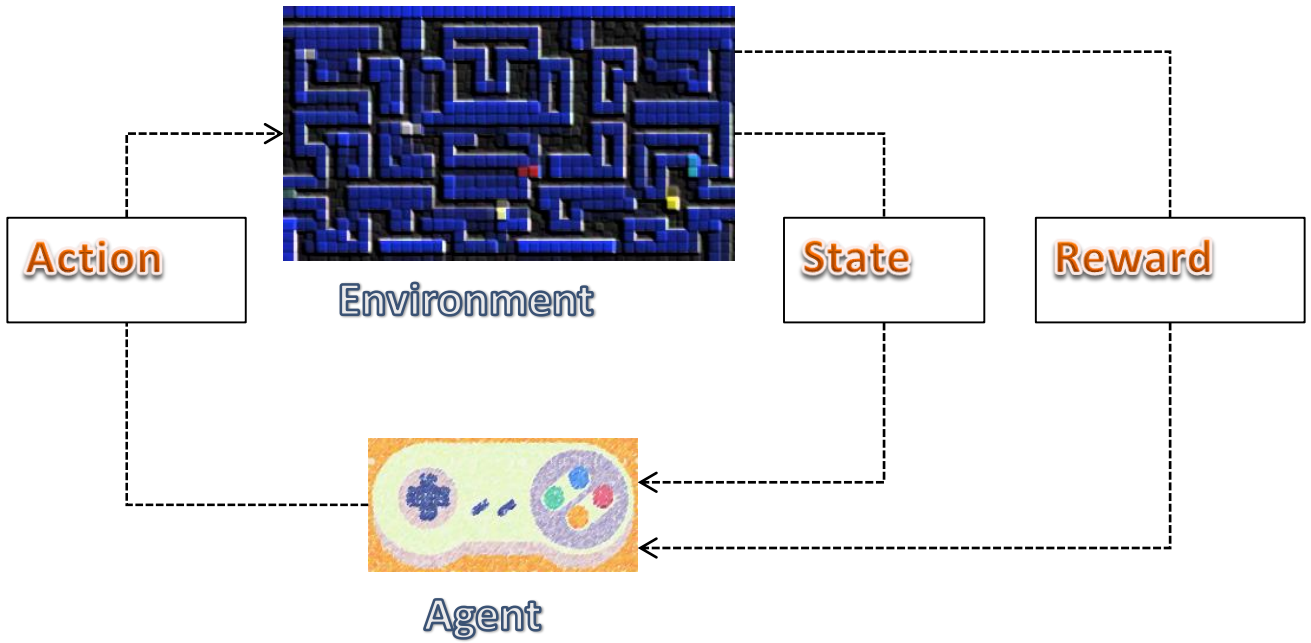
Reinforcement Learning (RL)

Prof. Muhammad Atif Tahir and Dr. Syed Ali Raza

NUCES Fast

Overview



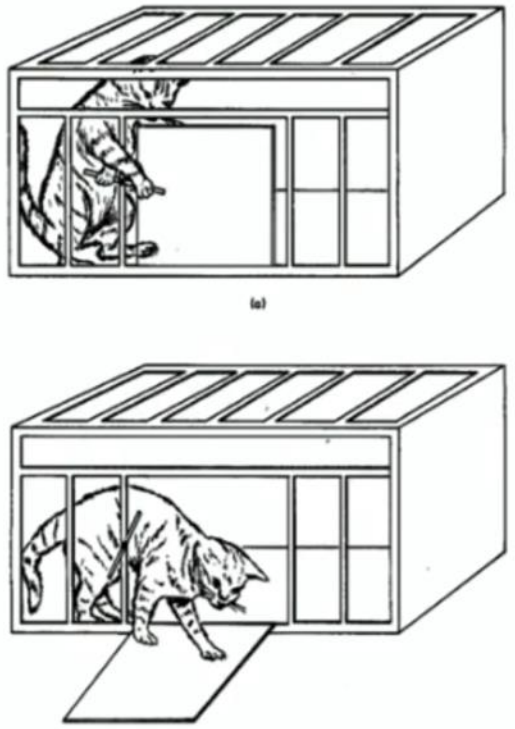
RL Introduction

- It solves sequential decision making problem (taking actions one after another to reach some goal from some start)
- The objective is to learn to act optimally in the environment
- After an action the learning agent receives a feedback in the form of rewards
- A learning agent’s utility is defined by the rewards
- Must (learn to) act so as to maximize expected rewards
- All learning is based on observed samples of outcomes

Brief History

- Roots in psychology and animal behaviour.
- He was a psychologist.
- The cat was motivated to go outside using some food.
- The cat would move around and eventually stumble on the method of releasing itself.
- In the subsequent trials, the cat would come out quickly.
- After some time, the cat would come out immediately.

Edward L. Thorndike (1874 –1949)

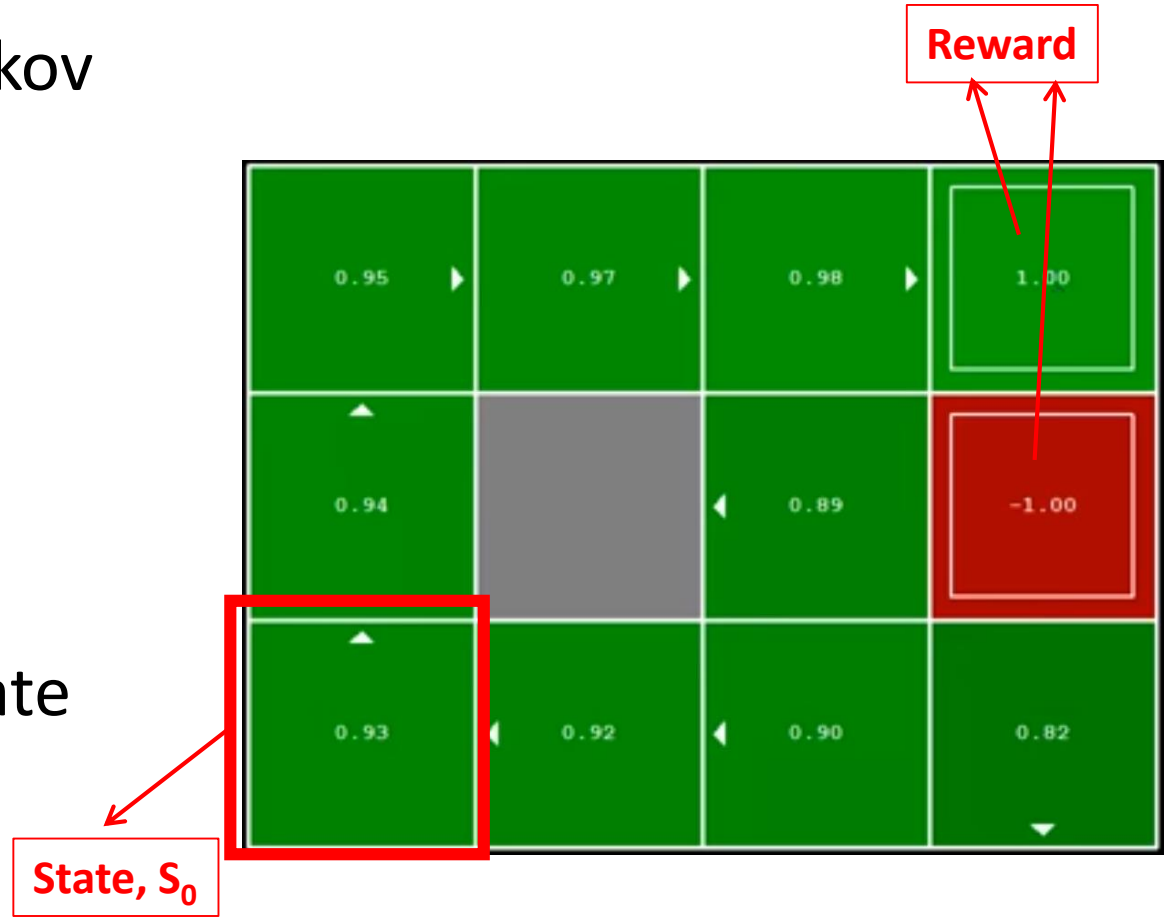


puzzle box

Learning by **Trial and Error**
Instrumental or operant Conditioning

Markov Decision Processes

- RL problem is defined based on Markov Decision Process (MDP)
- An MDP is defined by,
  - A set of states $s \in S$
  - A set of actions a $\in A$
  - Transition function $T(s, a, s')$
  - Reward function $\mathrm{R}(s, a, s')$
- Sometimes start state s_0 and end state become part of a MDP definition



What does Markov mean?

- "Markov" generally means that given the present state, the future and the past are independent

- In MDPs, "Markov" means the action outcomes depend only on the current state,

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots s_0)$$
$$=$$
$$P(s_{t+1}|s_t, a_t)$$

А. А. Марков (1886).

Policy

- In RL, we use a policy to decide a right to take from a state
- Its is a recommendation function which takes current state and tells what action to take, denoted as $\pi(s)$
- For MDPs, we want to achieve the optimal policy $\pi^*: S \rightarrow A$
- An optimal policy is one that maximizes expected utility if followed

Learning Optimality

- The goal of the learning agent (or the policy) is to collect as much rewards as possible.

'The optimality criteria is to maximize the rewards'

- If r_t is the reward it receives at time t, then the total reward is,

$$r_0 + r_1 + r_2 \ldots = \sum_{t=0}^{\infty} r_t$$

(Undiscounted utility)

Learning Optimality (Contd.)

- But, we discount the future rewards by multiplying them with a small number '*gamma*' ($0 \leq \gamma \leq 1$),

$$r_0 + \gamma r_1 + \ \gamma^2 r_2 \ldots = \sum_{t=0}^{\infty} \gamma^t \ r_t$$

(Discounted utility)

- γ is called the 'discount factor'
- The rewards which are farther away in time are highly discounted

Value Functions

- The utility of a sequence of rewards is defined as a value function
- A value function provides an estimation of goodness
- The degree of goodness can be defined for being in a state, $V(s)$, OR for taking an action a from state s, $V(s, a)$

Value Functions – State values

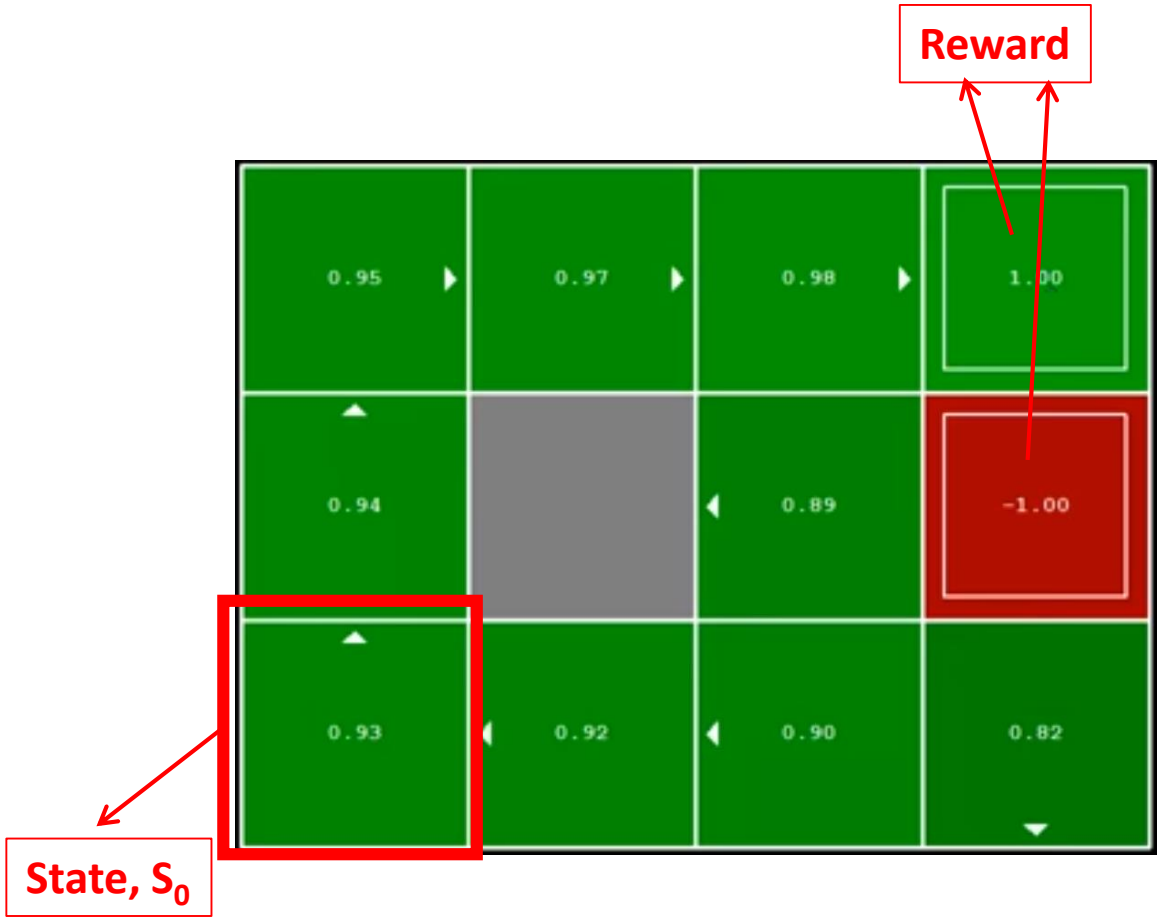
- Value for being in a state

$V(s_0) = 0.93$

$V(s_1) = 0.94$

$V(s_2) = 0.95$

…



Value Functions – Q-values

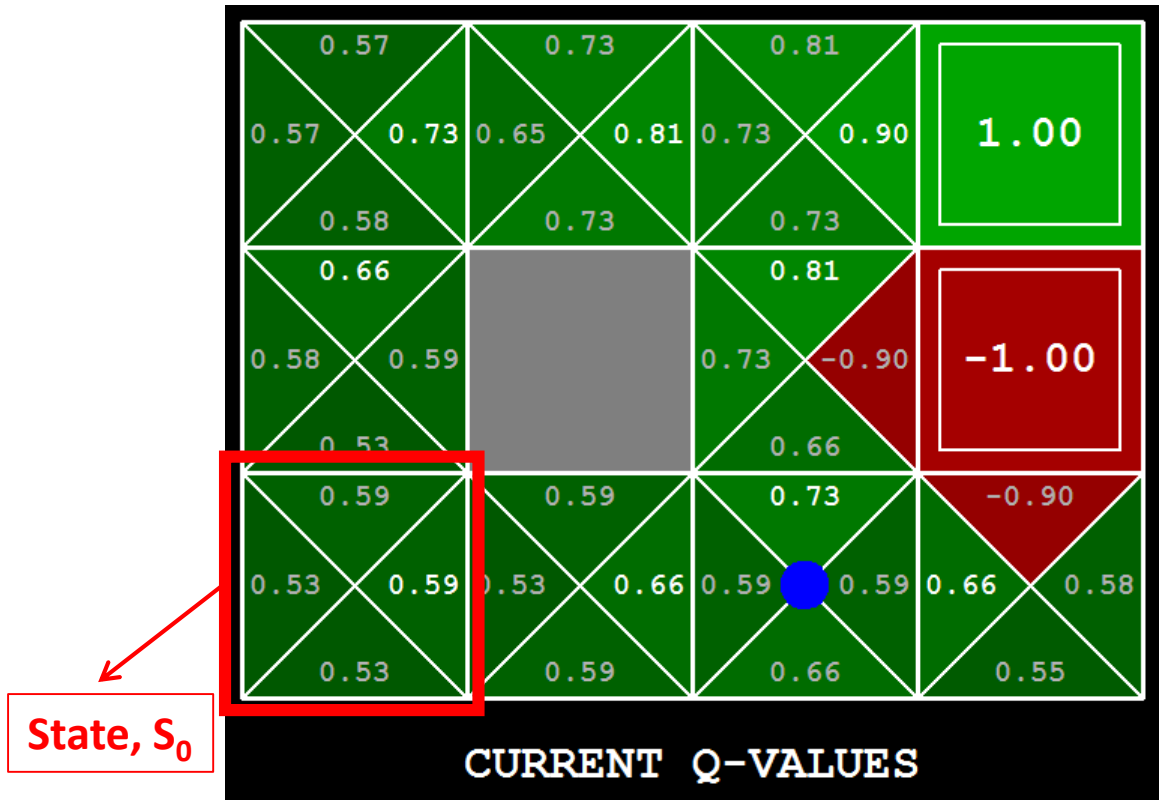
- $V(s, a)$ is also known as a Q-Value, $Q(s, a)$
- Value of taking action a from state s

$Q\ (s_0, Up) = 0.59$

$Q\ (s_0, Right) = 0.59$

$Q\ (s_0, Down) = 0.53$

$Q\ (s_0, Left) = 0.53$



Reinforcement Learning Methods

We call it reinforcement learning when $T\ and\ R$ (model) are unknown

RL methods mainly fall into two categories,

1. First learn the model, $T\ and\ R$, by iteratively interacting with the environment. Use $T\ and\ R$ to compute the policy using dynamic programming methods. This is referred to as model-based reinforcement learning.
2. Directly compute the value function from the interactions with the environment. This is referred to as model-free or direct reinforcement learning.

Q-Learning

- It is an online reinforcement learning algorithm
- It is based on the idea of adjusting the estimates of value function using the immediate reward and the discounted value of the immediate successor state
- The approach of updating value estimates using immediate experience is generally known as temporal difference learning

Q-Learning Update

- When an agent takes action a from s using its current policy,

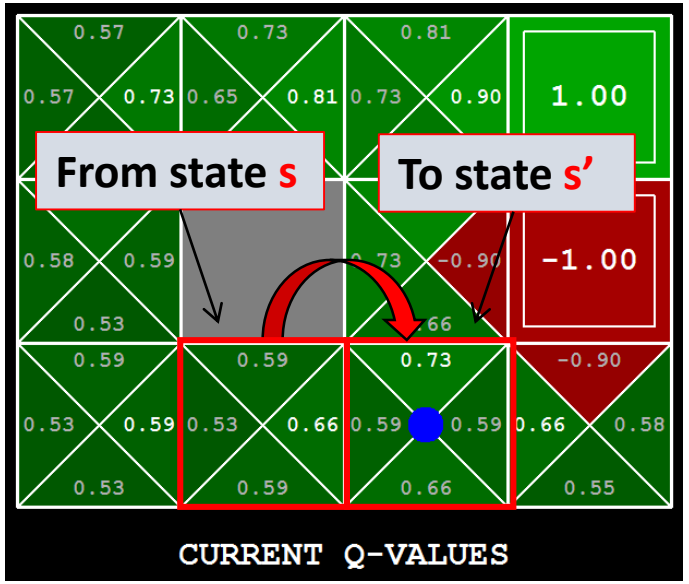
$$\pi \quad (s) = \arg\max_{a \in A} Q \quad (s, a),$$

it receives a reward r from the environment and reaches next state s'

- As a result of this transition, the agent has sampled a new experience from this environment

In this experience, it has received/discovered:

- a reward r
- and next state's Q-values, Q(s',a)



Q-Learning Update

VERY IMPORTANT

- Formally,
  - Sample = r + Q(s',a) ✗
  - Sample = r + γ max_a Q(s',a) ✓

  γ = discount factor

- Q-learning adds the sample to the old estimate of Q-value using a learning rate α as running average,

$$Q(s,a) = (\alpha)(sample) + (1-\alpha)Q(s,a)$$

$$Q(s,a) = (\alpha)\left[r + \gamma \max_a Q(s',a)\right] + (1-\alpha)Q(s,a)$$

More commonly, $$Q(s,a) = Q(s,a) + (\alpha)\left[r + \gamma \max_a Q(s',a) - Q(s,a)\right]$$

A step-by-step understanding of solving the Q-learning problem should be obtained from the example link provided below

https://people.revoledu.com/kardi/tutorial/ReinforcementLearning/Q-Learning-Example.htm

Q-Learning Algorithm

Algorithm 1 Q-learning as proposed by Watkins and Dayan (1992)

Require: discount factor γ, learning rate α

Initialize Q(s,a) arbitrarily
for all episodes **do**
    Initialize starting state s
    repeat
        Choose action a using Q and an exploration strategy
        Execute a
        Observe next state s' and recieve immediate reward r
        Update, $Q(s,a) \leftarrow Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)$
        $s := s'$
    until s' is not the terminal state
end for

Q-Learning

- Q-learning is classified as an off-policy learning.
- It learns values Q which directly approximate the optimal values Q∗, regardless of the policy being followed.
- Even if an agent acts randomly it will converge to an optimal policy
- The optimal policy is given by,

$$\pi^*(s) = \arg\max_{a \in A} Q^*(s, a)$$

Where, $Q^*(s, a)$ is an optimal Q-value

Q-Learning

- A policy will always converge to an optimal policy if,
  - Each state-action pair is visited an infinite number of times
  - The learning rate α decays in time, such that,

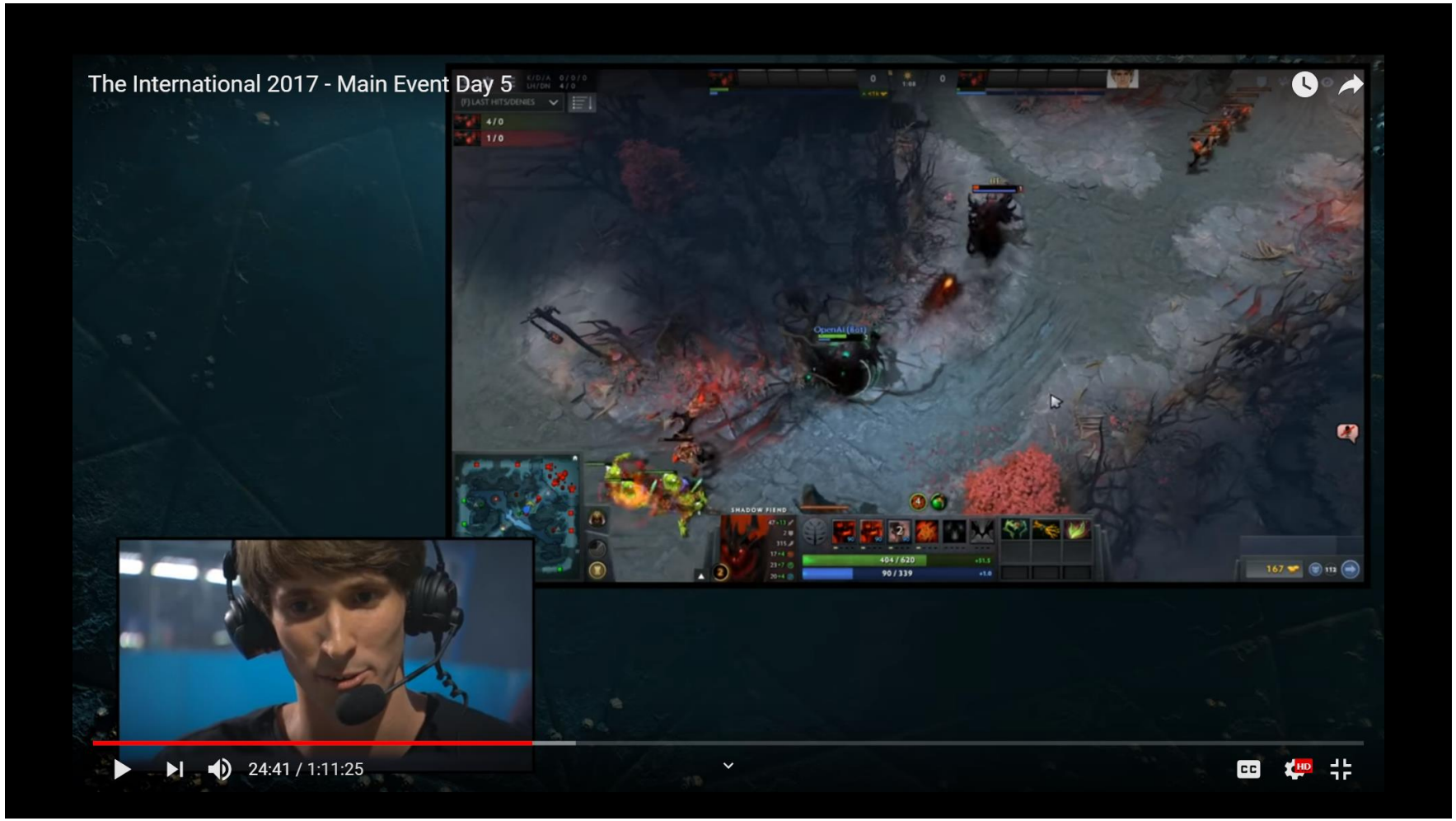
$$\sum_{t=0}^{\infty} \alpha_t = \infty \text{ and } \sum_{t=0}^{\infty} \alpha_t^2 < \infty$$

- In practice, it is commonly observed that the policy converges to an optimal policy much earlier, that is without infinite visits to every state-action
- In general, the policy can converge to an optimal policy much before the Q-values converge to optimal values.
- This makes it practically applicable to a wide range of problems

Exploration Strategy

- To sample from environment and to visit all the states in the environment, we need an exploration policy for action selection
- We need some randomness which can lead to more promising states
- Epsilon-greedy is the most commonly used exploration policy
  - With probability equal to epsilon (ϵ) take a random action and with probability $1 - \epsilon$ take the action with the highest Q-value

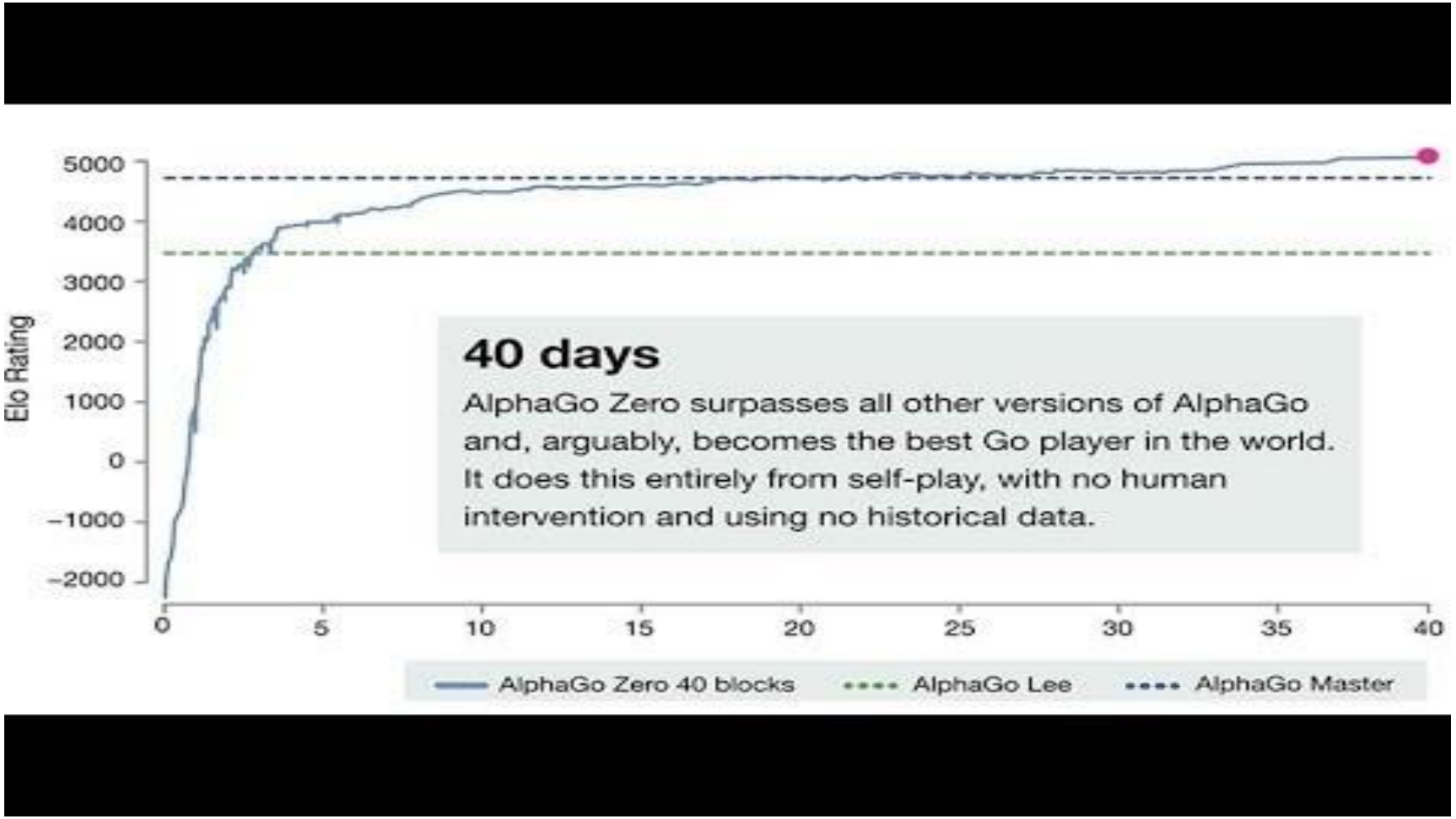
Success Stories – Dota 2



https://www.youtube.com/watch?time_continue=1242&v=yK5SxX3Ujs0

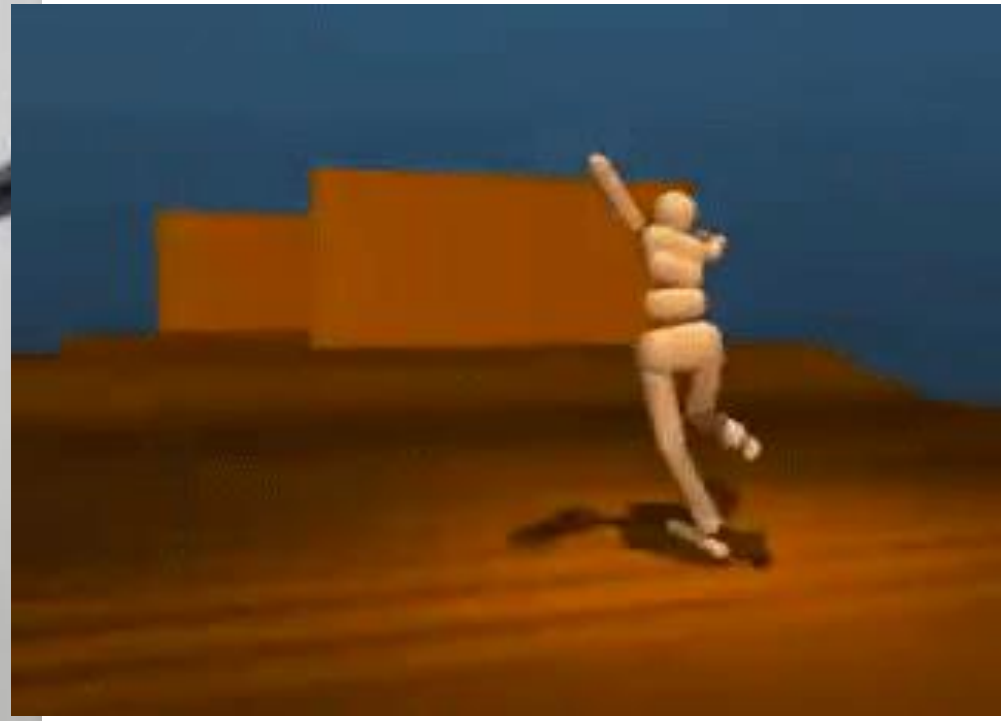
The International 2017 - Main Event Day 5

Success Stories – AlphaGo and Zero



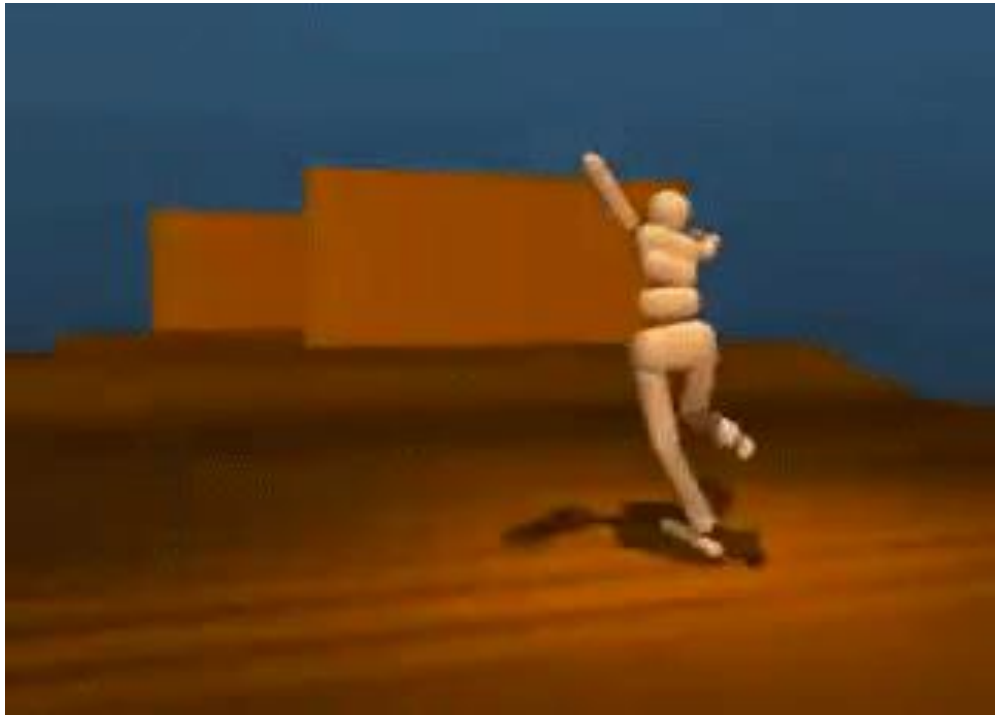
https://www.youtube.com/watch?time_continue=114&v=4Sm922Xp5N4
AlphaGo Zero - World's best Go player

Success Stories – Robotics I



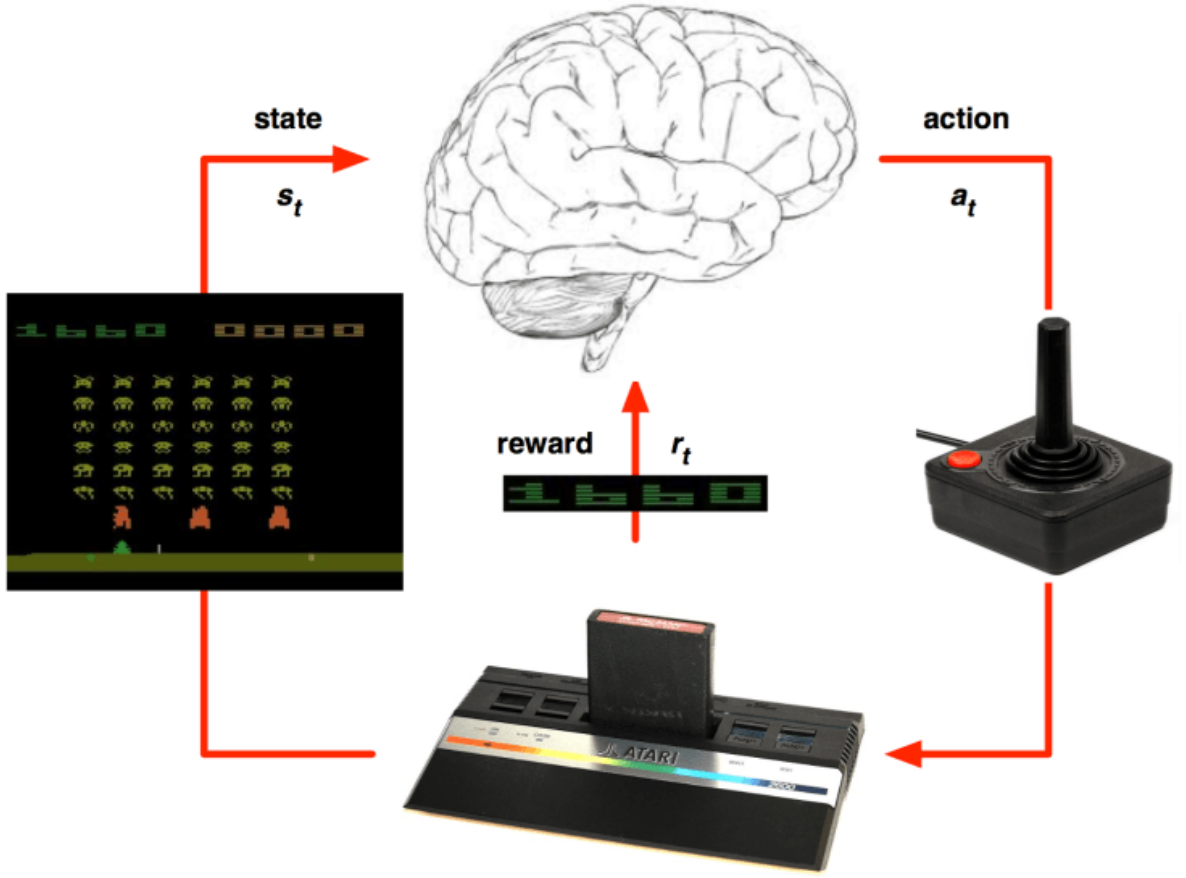
https://www.youtube.com/watch?v=jwSbzNHGflM
Learning Dexterity
https://www.youtube.com/watch?time_continue=138&v=hx_bgoTF7bs
Emergence of Locomotion Behaviours in Rich Environments

Success Stories – Robotics II



https://www.youtube.com/watch?time_continue=138&v=hx_bgoTF7bs
Emergence of Locomotion Behaviours in Rich Environments

Success Stories – DQN in Atari



https://deepmind.com/research/dqn/

Other Applications

- Resources management in computer clusters [1]
- Traffic Light Control [2]
- Web System Configuration [3]
- Chemistry [4]
- Personalized Recommendations [5]
- Bidding and Advertising [6]

[1] Mao, Hongzi, et al. "Resource management with deep reinforcement learning." *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*. ACM, 2016.
[2] Arel, Itamar, et al. "Reinforcement learning-based multi-agent system for network traffic signal control." *IET Intelligent Transport Systems* 4.2 (2010): 128-135.
[3] Bu, Xiangping, Jia Rao, and Cheng-Zhong Xu. "A reinforcement learning approach to online web systems auto-configuration." *2009 29th IEEE International Conference on Distributed Computing Systems*. IEEE, 2009.
[4] Zhou, Zhenpeng, Xiaocheng Li, and Richard N. Zare. "Optimizing chemical reactions with deep reinforcement learning." *ACS central science* 3.12 (2017): 1337-1344.
[5] Zheng, Guanjie, et al. "DRN: A deep reinforcement learning framework for news recommendation." *Proceedings of the 2018 World Wide Web Conference on World Wide Web*. International World Wide Web Conferences Steering Committee, 2018.
[6] Jin, Junqi, et al. "Real-time bidding with multi-agent reinforcement learning in display advertising." *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. ACM, 2018.

Acknowledgement

- Some of the slides were adopted from the course by Dan Klein and Pieter Abbeel offered at UC Berkeley, CS188 Intro to AI